Machine Learning (2026) 115:241
https://doi.org/10.1007/s10994-026-07170-3

TACCO: Actor–Critic Contrastive Optimization for OOD Action Mitigation in Offline Reinforcement Learning

Haotian Zhang[1] · **Chen Wang**[1] · **Jun Moon**[1]

Received: 1 October 2025 / Revised: 16 April 2026 / Accepted: 14 September 2026
© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract
Offline Reinforcement Learning provides a viable solution for tasks where online interaction is costly or infeasible by leveraging pre-collected data for policy learning. However, in an offline setting, the policy improvement process is prone to value overestimation due to distribution shifts, which can lead to policy degradation. Existing methods primarily address this issue through policy constraints or conservative value regularization, but few explicitly model the problem actions from a representation learning perspective. This paper proposes TACCO (TD3+BC with Actor-Critic Contrastive Optimization), a contrastive offline reinforcement learning method based on the TD3+BC framework, designed to address the out-of-distribution (OOD) action problem in offline reinforcement learning. We define OOD actions as those that lack local data support in a given state, do not match the current state, or exhibit low reliability. Furthermore, for dense and sparse reward environments, we design two distinct contrastive learning policies: TACCO-A performs contrastive learning on the actor side between data-driven actions, random actions, and cross-state policy actions; TACCO-B filters reliable samples on the critic side based on high Q-values and low uncertainty, and applies contrastive constraints to both reliable and remaining samples simultaneously. In this paper, we compare TACCO with six state-of-the-art offline reinforcement learning algorithms on the Mujoco and Adroit environments within the D4RL benchmark. Experimental results demonstrate that TACCO achieves superior performance on most tasks, validating its effectiveness in mitigating OOD action interference and enhancing policy stability.

Keywords Offline reinforcement learning · Contrastive learning · Out-of-distribution · D4RL

Haotian Zhang and Chen Wang have contributed equally to this work.

Editor: Peng Zhao.

Extended author information available on the last page of the article

Published online: 07 October 2026

1 Introduction

Deep reinforcement learning (DRL) has achieved significant progress across multiple domains including robot control, gaming, and recommendation systems (Tang et al. 2025; Li 2017; Afsar et al. 2022; Geng et al. 2025; Remmerden et al. 2025). However, deploying DRL in real-world scenarios remains challenging due to the high cost and potential risks associated with online interactions (Dulac-Arnold et al. 2019). Offline reinforcement learning (RL), as an emerging paradigm, offers a viable approach for safe and efficient learning by leveraging pre-collected datasets for policy learning without requiring further interaction with the environment (Levine et al. 2020).

However, offline RL still suffers from an out-of-distribution (OOD) problem. Since the policy may select actions that are rarely observed in the dataset, the corresponding value estimates are often unreliable. This bias leads to extrapolation errors and severe overestimation of Q-values, resulting in learning instability or even policy collapse (Levine et al. 2020; Fujimoto et al. 2019). To mitigate this issue, existing approaches primarily focus on policy constraints (Beeson and Montana 2022; Ma et al. 2024; Beeson and Montana 2024) or conservative value function regularization (Kumar et al. 2020; Liu et al. 2025). While these methods have achieved some progress, they mostly operate at the optimization level or value function level, often ignoring the representation problem of OOD actions.

In recent years, representation learning has achieved remarkable accomplishments in the field of deep learning, demonstrating formidable capabilities in tasks such as computer vision (Bengio et al. 2013), natural language processing (Devlin et al. 2019), and speech recognition (Baevski et al. 2020). By learning high-level features in latent spaces, models can better capture the intrinsic structure of input data, thereby enhancing generalization performance and task adaptability (Bengio et al. 2013; Bengio 2013). Among these, contrastive learning, as a self-supervised representation learning method, has garnered extensive attention in recent years. By constructing positive–negative sample pairs and pulling similar samples closer while pushing dissimilar ones farther apart in the latent space, it learns more discriminative and robust feature representations (Hu et al. 2024. Such methods have achieved breakthrough results in scenarios like image recognition and representation transfer, gradually extending into the DRL domain (Chen et al. 2020; Grill et al. 2020; Laskin et al. 2020). Thus, from a representation learning perspective, explicitly distinguishing OOD actions in the latent space offers a novel approach to addressing OOD challenges in offline RL. By learning state-action representations, constraints can be directly imposed to reduce the probability of selecting OOD actions and enhance policy stability. However, compared to optimization-based research, exploration in this direction remains relatively limited.

To this end, this paper proposed an offline RL algorithm called TD3+BC (Fujimoto and Gu 2021) with Actor-Critic Contrastive Optimization (TACCO) based on the TD3+BC framework, integrating contrastive learning into the state-action representation space to enhance the model's ability to distinguish OOD actions from in-distribution actions. Specifically, in a dense reward environment, we combine contrastive learning with the actor network to distinguish between state-action pairs in the dataset and OOD state-action pairs as defined in this paper, thereby steering the policy output away from OOD actions and improving the stability of the training process. In contrast, in sparse reward environments, we incorporate contrastive learning into the critic network, enabling it to more effectively distinguish high-Q, low-uncertainty state-action pairs from other samples, thereby enhancing the reliability

of the critic's estimates and better guiding policy learning. Experimental results demonstrate that this method significantly outperforms multiple existing state-of-the-art algorithms on both Mujoco and Adroit tasks within the D4RL benchmark, validating the effectiveness and potential of contrastive representation learning in mitigating OOD challenges within offline RL. In summary, the main contributions of this paper are as follows:

1. We proposed a novel offline RL algorithm named TACCO. Based on the TD3+BC framework, it incorporates contrastive learning to suppress OOD actions in the policy, thereby enhancing policy performance and stability.
2. Our method applies contrastive learning to the representation space of the actor or critic. By pulling in-distribution state-action pairs or high-Q, low-uncertainty state-action pairs closer in the semantic space, the algorithm effectively guides the policy toward more stable decisions.
3. On the Mujoco and Adroit environments of the D4RL benchmark, our approach achieves results close to, and in several environments surpasses, the state-of-the-art performance of current offline RL algorithms. Specifically, it demonstrates near-universal superiority across the Mujoco suite of tasks and remains competitive in more challenging scenarios like Adroit, consistently securing top-two rankings in the majority of overall results.

2 Related Work

2.1 Offline Reinforcement Learning (Offline RL)

The core objective of offline RL is to optimize policies solely based on a static dataset without additional interactions with the environment. This paradigm offers significant safety and efficiency in practical applications by avoiding costly or high-risk online sampling processes. Typically, we model the environment as a Markov Decision Process (MDP):

$$M = \{\mathcal{S}, \mathcal{A}, P, r, \gamma\},$$

where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ denotes the action space, $P(s' \mid s, a)$ is the state transition function, $r(s, a)$ is the immediate reward, and $\gamma \in [0, 1]$ is the discount factor.

At each time step t, the agent selects an action a_t according to policy $\pi_\theta(a|s)$, receives reward r_t, and transitions to the next state s_{t+1}. Its cumulative discounted return is expressed as $R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$. The objective of offline RL is to learn the optimal policy π_θ^* from the dataset $D = \{(s_t, a_t, r_t, s_{t+1})\}_{t=0}^{T}$, so as to maximize the expected return. Therefore, offline RL faces a series of unique challenges:

- OOD problem: Policies may generate state-action pairs unseen in the training dataset, leading to overestimation of the value function and policy collapse.
- Exploration constraints: Without direct interaction with the environment, offline RL struggles to compensate for data scarcity through active exploration as online methods do.

Consequently, recent research has focused on proposing improved approaches to mitigate these issues through policy constraints, value function regularization, uncertainty estimation, and other techniques (Levine et al. 2020).

2.2 Contrastive Learning

Contrastive learning as an important method in recent years in the field of representation learning centers on the core idea of learning the latent structure of data through "comparison." Unlike traditional generative methods, contrastive learning does not directly reconstruct inputs. Instead, it constructs representations capable of capturing semantic invariance and discriminative properties by pulling positive samples (similar samples) closer together and pushing negative samples (dissimilar samples) farther apart in the representation space (Hu et al. 2024). At the representation level, contrastive learning excels by directly shaping the geometric structure of the embedding space. This enables representations that not only maintain local smoothness but also form global semantic clusters (Le-Khac et al. 2020). Consequently, representations derived from contrastive learning often exhibit superior generalization and transferability, particularly when labeled data is scarce for downstream tasks. For instance, frameworks based on InfoNCE loss effectively prevent representation collapse by maximizing consistency among positive samples and separability from negative samples (He et al. 2020); SupCon (Supervised Contrastive Learning) extends contrastive learning to supervised settings, imposing stronger aggregation constraints on same-category samples under category label guidance, thereby significantly improving classification performance (Khosla et al. 2020). Compared to generative self-supervised methods, contrastive learning exhibits greater "discriminative" properties, enabling robust modeling capabilities across high-dimensional domains such as vision, language, and graph structures. This demonstrates that contrastive learning transcends mere feature extraction, serving as a representation mechanism deeply integrated with decision optimization.

2.3 Contrastive Learning in Reinforcement Learning

Contrastive representation learning has recently emerged as an effective paradigm to enhance DRL by improving the quality of state and action embeddings. Early work such as CURL leverages contrastive loss to extract compact visual features from raw pixels, achieving superior sample efficiency in pixel-based control tasks (Laskin et al. 2020). Building upon this, M-CURL incorporates masked pretraining and Transformer modules to exploit correlations among consecutive frames, further boosting representation quality in sequential environments (Zhu et al. 2022). More fundamentally, contrastive learning can be reinterpreted as goal-conditioned RL itself, where the similarity between state-action and future state embeddings corresponds to a value function (Eysenbach et al. 2022). Following this line, stable contrastive RL demonstrates how careful design choices (e.g., encoder architectures, normalization, data augmentation) yield robust goal-reaching policies and representations that capture causal relationships in robotic tasks (Zheng et al. 2023). In offline RL, contrastive objectives have also been employed to mitigate distributional challenges: for example, Contrastive Diffuser pulls trajectories toward high-return states and pushes them away from low-return states, making effective use of suboptimal data (Shan et al. 2024). Collectively, these studies indicate that contrastive representation learning not only enhances sample effi-

ciency but also provides a principled way to align DRL embeddings with control objectives, thereby improving policy stability and generalization across diverse domains.

Compared to the CURL algorithm described above, TACCO extends its contrastive learning scope to state-action pairs rather than limiting it to state representations alone. Furthermore, its optimization objective differs fundamentally: TACCO aims to explicitly distinguish between data-supported actions and OOD-like actions, thereby addressing out-of-distribution challenges in offline reinforcement learning more directly. Furthermore, unlike offline RL methods that address OOD problems through conservative regularization or in-sample evaluation, TACCO adopts a representation learning approach to explicitly model and distinguish OOD behaviors in the embedding space. This contrasts with existing methods, which primarily operate at the level of the objective function or value estimation. In this way, TACCO constructs a more structured representation space, enabling it to effectively distinguish negative samples from positive ones. The relevant design and analysis are detailed in Sect. 3.3.

3 Method

3.1 TD3+BC

TD3+BC is an algorithm used in offline RL, representing a small yet significant refinement over the foundational TD3 algorithm (Fujimoto et al. 2018). The challenge in offline RL lies in the fact that agents can only learn from pre-collected datasets, lacking real-time interaction with the environment. This can lead to overestimation bias in Q-values (functions measuring action value), particularly when evaluating actions absent in the evaluation dataset. Such errors accumulate during training, ultimately causing learning failure.

To address this, TD3+BC incorporates a Behavior Cloning (BC) term during policy updates to regularize the policy. This encourages the agent to learn a policy closer to the behaviors observed in the dataset, thereby preventing inaccurate overestimation of unseen actions. The loss functions for the actor and critic in the TD3+BC algorithm are defined as follows:

$$\mathcal{L}_{\text{actor}} = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[-\lambda\, Q_\theta(s, \pi_\phi(s)) + \|\pi_\phi(s) - a\|^2\right], \tag{1}$$

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\|Q_{\theta_1}(s,a) - y\|^2 + \|Q_{\theta_2}(s,a) - y\|^2\right], \tag{2}$$

where the target value y is defined as

$$y = r + \gamma Q_\theta(s', \pi_\phi(s')), \tag{3}$$

with double Q-networks mitigating overestimation bias and target networks θ' and ϕ' ensuring training stability. Finally, the overall policy update objective of TD3+BC can be expressed as:

$$\pi = \underset{\pi}{\text{argmax}}\,\mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\lambda Q(s, \pi(s)) - (\pi(s) - a)^2\right], \tag{4}$$

$$\lambda = \frac{\eta}{\frac{1}{N}\sum_{(s_i,a_i)} |Q(s_i, a_i)|}, \tag{5}$$

Among these, λ is adjusted via hyperparameter η to balance the RL objective of maximizing Q-values against the behavior cloning objective. To maintain the effectiveness of λ across different tasks, TD3+BC normalizes Q-values, standardizing their numerical range. Simultaneously, state data is also normalized, making the algorithm easier to implement and reducing computational costs.

3.2 Supervised Contrastive Learning (SupCon)

Contrastive learning is a representation learning method whose core objective is to learn discriminative embedding spaces by constructing similarity relationships among samples. Within this framework, three concepts are typically introduced: anchors, positive samples, and negative samples. Anchors represent the samples currently under consideration, positive samples are those semantically or categorically similar to the anchor, while negative samples are the opposites of positive samples. In this algorithm, we adopt the supervised contrastive learning approach proposed by Khosla et al. (2020). We employ the following supervised contrastive loss:

$$\mathcal{L}^{\text{sup}} = \sum_{i \in I} \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\boldsymbol{z}_i \cdot \boldsymbol{z}_p / \tau)}{\sum_{a \in A(i)} \exp(\boldsymbol{z}_i \cdot \boldsymbol{z}_a / \tau)}, \tag{6}$$

where i denotes the anchor sample, I represents the set of sample indices for the entire batch, $P(i)$ denotes the set of positive samples belonging to the same category as anchor i, and $A(i)$ denotes the set of all samples excluding anchor i itself; z is the embedding representation obtained by passing the sample through the encoder; τ is the temperature coefficient, used to scale the similarity distribution and thereby adjust the model's sensitivity to difficult samples.

3.3 TD3+BC with Actor-Critic Contrastive Optimization (TACCO)

Require: Dataset $\mathcal{D}$, discount factor γ, target update rate τ, BC weight α, contrastive weight β, temperature τ_c
Ensure: Learned actor π_ϕ and critic Q_θ
Initialize critic θ, actor ϕ (backbone g_ϕ shared by policy head μ_ϕ and embedding head e_ϕ), target networks θ', ϕ'
for $t \leftarrow 0$ **to** T **do**
  Sample minibatch $(s, a, r, s', d) \sim \mathcal{D}$ ▷ Critic update
  $a' \leftarrow \text{clip}(\pi_{\phi'}(s') + \epsilon,\ -a_{\max}, a_{\max}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$
  $y \leftarrow r + \gamma(1-d)\min\big(Q_{\theta'_1}(s', a'),\ Q_{\theta'_2}(s', a')\big)$
  Update θ by minimizing $\mathcal{L}_{\text{critic}} = \sum_{i=1,2} \mathbb{E}\left[(Q_{\theta_i}(s,a) - y)^2\right]$
  if t mod policy_freq $= 0$ **then** ▷ Actor update with contrastive loss
    Sample $a_{\text{random}} \sim \text{Uniform}(-a_{\max}, a_{\max})$
    Compute $a_{\text{next}} \leftarrow \pi_\phi(s')$ ▷ Mismatched negative
    Construct contrastive pairs:
      Positives$^+$: $\{(s,a) \mid (s,a) \in \mathcal{D}\}$
      Negatives$^-$: $\{(s, a_{\text{random}})\} \cup \{(s, a_{\text{next}})\}$
    Compute embeddings via shared actor backbone:
      $z \leftarrow [f_\phi(s,a),\ f_\phi(s, a_{\text{random}}),\ f_\phi(s, a_{\text{next}})]$
    Compute SupCon loss by Eq. (6) and λ by Eq. (5)
    Update ϕ by minimizing Eq. (10)
    $\theta' \leftarrow \tau\,\theta + (1-\tau)\,\theta', \quad \phi' \leftarrow \tau\,\phi + (1-\tau)\,\phi'$
  end if
end for
return $\pi_\phi,\ Q_\theta$

Algorithm 1 TACCO-A

Require: Dataset $\mathcal{D}$, discount factor γ, target update rate τ, temperature τ_c, Q-threshold ρ_q, uncertainty threshold ρ_u, encoder layer update rate δ
Ensure: Learned actor π_ϕ and critic Q_θ
Initialize actor parameters ϕ and target ϕ'
Initialize critic parameters θ comprising shared encoder f_ψ and dual Q-heads $\{Q_{\theta_1}, Q_{\theta_2}\}$, and target θ'
for $t \leftarrow 0$ **to** T **do**
  Sample minibatch $(s, a, r, s', d) \sim \mathcal{D}$ ▷ — Critic update —
  $a' \leftarrow \text{clip}(\pi_{\phi'}(s') + \epsilon,\ -a_{\max}, a_{\max}),\quad \epsilon \sim \mathcal{N}(0, \sigma)$
  $y \leftarrow r + \gamma(1-d)\min\big(Q_{\theta'_1}(s', a'),\ Q_{\theta'_2}(s', a')\big)$
  Update θ by minimizing $\mathcal{L}_{\text{critic}} = \sum_{i=1,2} \mathbb{E}\Big[\big(Q_{\theta_i}(s,a) - y\big)^2\Big]$
  if t mod policy_freq $= 0$ **then** ▷ — Actor update —
    $\lambda \leftarrow \frac{\eta}{\frac{1}{N}\sum_{(s_i,a_i)} |Q(s_i, a_i)|}$
    Update ϕ by minimizing Eq. (1)
    Soft-update target networks:
      $\phi' \leftarrow \tau\phi + (1-\tau)\phi'$
      $\psi' \leftarrow \delta\psi + (1-\delta)\psi'$ ▷ encoder uses slightly faster update rate
      $\theta'_i \leftarrow \tau\theta_i + (1-\tau)\theta'_i,\ i = 1, 2$
  end if
  Compute $\pi_\phi(s)$, $Q_{\theta_1}(s, \pi_\phi(s))$, $Q_{\theta_2}(s, \pi_\phi(s))$ **without** gradient ▷ — Contrastive encoder update —
  Select top-ρ_q% and top-ρ_u%
  Construct positive/negative pairs.
  $z \leftarrow f_\psi\big(\big[\text{Positives}^+ \cup \text{Negatives}^-\big]\big)$ ▷ Encode via shared encoder
  Compute SupCon loss
  Update encoder ψ only by minimizing $\mathcal{L}^{\text{sup}}$
end for
return $\pi_\phi,\ Q_\theta$

Algorithm 2 TACCO-B

To explicitly constrain the representation space of in-distribution actions and out-of-distribution (OOD) actions in offline reinforcement learning, we introduce supervised contrastive learning on top of TD3+BC. In our implementation, the contrastive encoder is not a completely separate network; instead, it is constructed from part of the actor-critic network and shares parameters with either the actor or the critic depending on the reward structure of the environment. Specifically, we design two variants:

1. **TACCO-A**: for dense-reward environments, where contrastive learning is introduced on the actor side for representation learning;
2. **TACCO-B**: for sparse-reward environments, where contrastive learning is introduced on the critic side for representation learning.

This differentiated design is motivated by prior studies (Hare 2019; Han and Sung 2021; Wilcox et al. 2022; Matheron et al. 2019; Yang et al. 2026) which suggest that in dense-reward settings, the critic is generally easier to train and can provide stable value estimates, allowing direct optimization of the actor to effectively improve policy performance. In contrast, in sparse-reward environments, the optimization of the critic becomes crucial, as the stable and high-quality value estimates provided by the critic can offer reliable gradient signals, thereby guiding the actor toward optimal solutions. The experimental results in

Sect. 4.3.1 also support the above theory and Fig. 1 illustrates the overall pipeline of these two variants.

In addition, we partition all rewards in the environment. Specifically, we normalize the rewards to [0,1] and discretize them into K (K=50) intervals, with the fill rate expressed as

$$\text{FillRate} = \frac{1}{K}\sum_{i=1}^{K} 1\left(n_i > 0\right), \tag{7}$$

where n_i is the sample count of the i-th bin, a lower fill rate indicates that rewards concentrate in a small subset of bins, reflecting sparse reward signals. We set the fill rate to 0.3, meaning that environments with a fill rate below 0.3 are defined as sparse environments. Table 1 shows the fill rates for the Mujoco and Adroit environments.

3.3.1 TACCO-A: Contrastive Constraint Based on Actor Representations

In TACCO-A, the actor consists of the following three components:

- **Actor backbone** $g_\phi : s \mapsto h(s)$;
- **Policy output head** $\mu_\phi : h(s) \mapsto \pi_\phi(s)$;
- **Contrastive embedding head** $e_\phi : [h(s), a] \mapsto z$.

That is, the actor first maps the state s into a hidden representation $h(s)$, and then outputs the policy action through the action head. For the contrastive learning part, we reuse the same hidden state representation $h(s)$, concatenate it with a candidate action a, and feed the result into the contrastive embedding head to obtain the contrastive embedding z. Therefore, in TACCO-A, the shared part is the state-encoding backbone of the actor (Actor backbone), while the policy output head and the contrastive embedding head are separated from each other. This design allows contrastive learning to influence the state representation network, thereby suppressing the tendency of the actor to generate unfamiliar or OOD actions. More detailed network configurations are provided in the Appendix A.

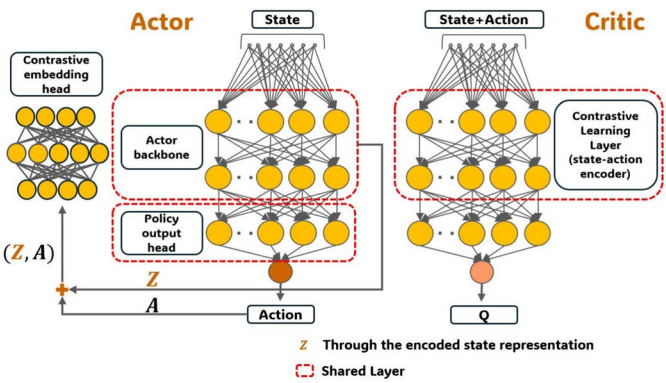


Fig. 1 Algorithmic flowchart of TACCO (Left: TACCO-A; Right: TACCO-B). In this flow, state-action pairs have been explicitly categorized into positive and negative samples

Table 1 Reward fill rate for all D4RL environments ($K = 50$ bins, $\delta = 0.3$)

Environment	fill rate
Halfcheetah-random-v2	1.00
Halfcheetah-medium-v2	1.00
Halfcheetah-medium-replay-v2	1.00
Halfcheetah-medium-expert-v2	1.00
Halfcheetah-expert-v2	1.00
Hopper-random-v2	1.00
Hopper-medium-v2	1.00
Hopper-medium-replay-v2	1.00
Hopper-medium-expert-v2	1.00
Hopper-expert-v2	1.00
Walker2d-random-v2	1.00
Walker2d-medium-v2	1.00
Walker2d-medium-replay-v2	1.00
Walker2d-medium-expert-v2	1.00
Walker2d-expert-v2	1.00
Door-human-v1	0.08
Door-cloned-v1	0.08
Door-expert-v1	0.08
Hammer-human-v1	0.12
Hammer-cloned-v1	0.14
Hammer-expert-v1	0.12
Relocate-human-v1	0.12
Relocate-cloned-v1	0.12
Relocate-expert-v1	0.08
Pen-human-v1	0.08
Pen-cloned-v1	0.14
Pen-expert-v1	0.14

Environments with fill rate $< \delta$ are classified as sparse-reward

Based on the above design objective, we treat the negative samples that should be pushed away as OOD-like state–action pairs. Accordingly, we perform contrast at the level of state–action pairs. For a given state s, the dataset action a is regarded as an in-distribution positive sample that is compatible with the state. Meanwhile, we construct two types of negative samples: a random action a_{random} and a policy action $\pi_\phi(s')$ generated at the next state. Specifically, a_{random} serves as a simple off-support negative, while $\pi_\phi(s')$, although generated by the policy network, forms a deliberately mismatched state–action pair with the current state s, and is therefore treated as a harder negative.

The construction of these OOD-like state–action pairs is consistent with the conservative principles adopted in CQL (Kumar et al. 2020; MCQ Lyu et al. 2022, and PBRL Bai et al. 2022) for handling actions with insufficient data support. Specifically, MCQ points out that in Bellman backup, actions sampled from the learned policy may fall outside the support of the behavior policy, leading to arbitrarily erroneous value estimates. PBRL further argues that the greedy action selected by the target Q-network at the next state can be considered OOD, since the corresponding state–action pair is rarely covered by the dataset, and thus explicitly regularizes such OOD samples constructed from dataset states and policy-generated actions. CQL also shows that offline RL suffers from action distribution shift, and that uniformly sampled random actions can serve as representative OOD candidates when

analyzing and suppressing overestimation. Thus, the OOD state-action pairs in TACCO-A are defined as follows:

- (s, a_{random}): randomly paired actions that are not supported by the local data distribution under state s;
- $(s, \pi_\phi(s'))$: mismatched pairs where the action originates from a different state and is incompatible with the current state s.

We further evaluate (s, a), (s, a_{random}), and $(s, \pi_\phi(s'))$ in the hopper-medium-replay environment by measuring their distances to the locally supported action region under each state. Specifically, for each state s, we perform a K-nearest neighbors (KNN) search in the dataset state space to retrieve its K nearest neighboring states (with $K = 5$). The actions corresponding to these neighboring states can be regarded as the set of actions supported by the local data near that state. For a given action a (data action $a_{\mathcal{D}}$, random action a_{random} and policy action $\pi_\phi(s')$), we compute the minimum ℓ_2 distance between it and the actions corresponding to neighboring states. To account for scale differences across states, we normalize actions of different types relative to the average distance of actions in the dataset, defining the following ratio metric:

$$\text{ScoreRatio}(a) = \frac{\mathbb{E}_s\left[\min_{i\in\mathcal{N}_K(s)} \left\|a - a_{\mathcal{D},i}\right\|_2\right]}{\mathbb{E}_s\left[\min_{i\in\mathcal{N}_K(s)} \left\|a_{\mathcal{D}} - a_{\mathcal{D},i}\right\|_2\right]}. \tag{8}$$

A larger ratio indicates that the action deviates further from the local data support region compared to dataset actions.

The Fig. 2 shows that both random actions and $\pi_\phi(s')$ are farther from the local data support than dataset actions, with score ratios of approximately 3 times further and 1.6 times further, respectively, which explains their use as off-support or mismatched negatives. We define the positive and negative pairs for contrastive learning in TACCO-A as follows:

$$\begin{aligned} Positives^{+}_{actor} &= \{(s,a) \mid (s,a) \in \mathcal{D}_{D4RL}\}, \\ Negatives^{-}_{actor} &= \{(s, a_{\text{random}})\} \cup \{(s, \pi_\phi(s'))\}. \end{aligned} \tag{9}$$

Accordingly, the actor optimization objective of TACCO-A can be written as:

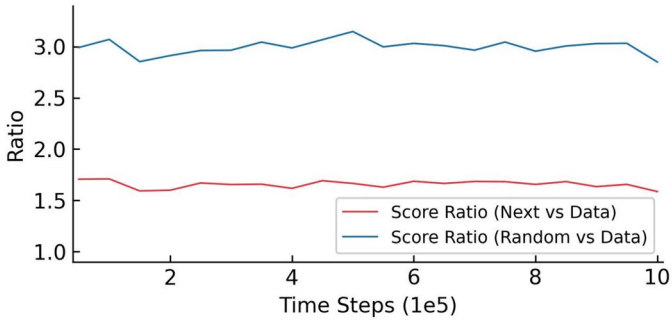


Fig. 2 Score ratio

$$\mathcal{L}_{\text{actor}}^{\text{TACCO-A}} = -\lambda \mathbb{E}_{s\sim\mathcal{D}}\left[Q_1\left(s, \pi_\phi(s)\right)\right] + \alpha \mathbb{E}_{(s,a)\sim\mathcal{D}} \left\|\pi_\phi(s) - a\right\|^2 + \beta \mathcal{L}^{\text{sup}}, \tag{10}$$

where λ follows the original TD3+BC setting, α denotes the weight of the behavior cloning term, and β denotes the weight of the contrastive loss.

3.3.2 TACCO-B: Contrastive Constraint Based on Critic Representations

In TACCO-B, the contrastive encoder is placed inside the critic. Specifically, the critic is decomposed into two parts:

- **Shared state–action encoder** f_ψ;
- **Q network**.

That is, the encoder f_ψ is shared by both Q-function branches as well as by the contrastive objective. This design enables the critic to distinguish and judge different types of positive and negative samples more precisely through the contrastive learning mechanism. Inspired by An et al. (2021), we treat Q-values and uncertainty as a joint criterion. Specifically, An et al. (2021) points out that in offline RL, overestimation of Q-values often stems from uncertain predictions of OOD state-action pairs, and this uncertainty can be characterized by the divergence of the Q-ensemble. That is, state-action pairs within the data distribution typically exhibit lower Q-value variance, while out-of-distribution or under-supported actions exhibit higher uncertainty.

Based on this, we construct the contrastive samples as follows:

$$\begin{aligned} Positives_{critic}^{+} = \{(s,a) \mid & Q(s,a) \text{ is in high Q-values,} \\ & U(s,a) \text{ is in low uncertainty} \rho_u\}, \\ Negatives_{critic}^{-} = \{(s,a) \mid & (s,a) \notin Positives_{critic}^{+}\}, \end{aligned} \tag{11}$$

Within each batch, we divide all $(s, \pi(s))$ pairs into $Positives_{critic}^{+}$ and $Negatives_{critic}^{-}$. The $Positives^{+}$ consist of those $(s, \pi(s))$ pairs in the batch that simultaneously satisfy high Q-values and low uncertainty; the $Negatives^{-}$ are all remaining $(s, \pi(s))$ pairs in the batch except for the $Positives^{+}$.

Concretely, the high Q-values is defined based on the mean estimate of the double Q-networks for the policy action $\pi(s)$:

$$Q_{\text{mean}}(s, \pi(s)) = \frac{1}{2}\left(Q_1(s, \pi(s)) + Q_2(s, \pi(s))\right). \tag{12}$$

We rank all $(s, \pi(s))$ pairs according to $Q_{\text{mean}}(s, \pi(s))$ in descending order and select the top $\rho_q\%$ samples as a candidate set. On top of this, we introduce an uncertainty measure to filter out unreliable critic estimates. Specifically, we quantify uncertainty as the absolute difference between the two Q-network outputs:

$$U(s, \pi(s)) = \left|Q_1(s, \pi(s)) - Q_2(s, \pi(s))\right|. \tag{13}$$

Table 2 The table compares the performance of TACCO against six baseline algorithms on the Mujoco (`-v2`) and Adroit (`-v1`) environments within the D4RL benchmark

Task environment	CQL	TD3+BC	DT	IQL	DTQL	O-DICE	TACCO (Ours)
Halfcheetah-m-r	44.70 ± 0.29	44.27 ± 0.54	39.89 ± 0.88	44.27 ± 0.15	50.47 ± 0.46	42.37 ± 2.14	**52.36 ± 0.98**
Halfcheetah-m	47.09 ± 0.19	48.54 ± 0.32	42.67 ± 0.31	48.40 ± 0.06	57.78 ± 0.38	47.12 ± 0.40	**58.34 ± 0.42**
Halfcheetah-m-e	**94.59 ± 0.92**	91.81 ± 3.24	92.39 ± 0.65	94.36 ± 0.45	93.26 ± 0.26	87.53 ± 4.58	94.13 ± 1.64
Halfcheetah-e	95.81 ± 0.33	97.15 ± 0.60	92.80 ± 0.44	96.85 ± 0.22	94.16 ± 0.44	5.63 ± 1.14	**97.77 ± 0.42**
Halfcheetah-r	29.31 ± 2.16	11.91 ± 0.57	1.97 ± 0.05	11.58 ± 4.85	24.85 ± 0.31	3.26 ± 1.55	**29.39 ± 1.48**
Hopper-m-r	93.13 ± 9.01	51.01 ± 9.20	81.15 ± 7.63	93.86 ± 7.02	99.58 ± 1.03	87.17 ± 15.47	**101.31 ± 0.86**
Hopper-m	60.33 ± 1.91	56.80 ± 1.54	62.93 ± 2.59	59.69 ± 2.47	**97.73 ± 5.45**	52.83 ± 6.57	70.52 ± 5.04
Hopper-m-e	110.60 ± 1.43	103.95 ± 9.66	**111.11 ± 0.51**	59.17 ± 18.85	110.73 ± 1.95	101.37 ± 8.22	109.48 ± 2.04
Hopper-e	108.83 ± 2.51	109.96 ± 1.81	111.08 ± 0.23	104.67 ± 4.36	105.96 ± 3.30	2.53 ± 0.20	**112.29 ± 0.43**
Hopper-r	13.06 ± 8.15	8.47 ± 0.84	7.61 ± 1.46	7.26 ± 0.09	7.77 ± 0.62	2.35 ± 0.11	**31.70 ± 0.43**
Walker2d-m-r	76.24 ± 6.07	73.97 ± 14.83	68.89 ± 4.84	81.06 ± 4.35	85.91 ± 6.24	61.81 ± 4.61	**89.25 ± 1.50**
Walker2d-m	83.30 ± 0.47	84.60 ± 1.38	75.48 ± 1.83	81.03 ± 4.43	**88.47 ± 0.64**	83.49 ± 1.50	85.07 ± 2.14
Walker2d-m-e	109.77 ± 0.25	110.34 ± 0.56	108.50 ± 0.31	111.58 ± 0.53	110.24 ± 0.33	109.97 ± 0.29	**111.61 ± 0.24**
Walker2d-e	110.01 ± 0.19	110.27 ± 0.06	108.05 ± 0.13	**112.59 ± 0.17**	111.65 ± 0.28	31.12 ± 29.92	111.84 ± 0.44
Walker2d-r	0.21 ± 0.72	3.02 ± 1.88	4.37 ± 3.39	4.18 ± 1.75	11.73 ± 3.24	8.18 ± 2.51	**12.77 ± 6.14**
Mujoco total	966.88	906.07	888.88	910.55	1050.33	626.74	**1187.47**
Pen-c	12.50 ± 3.36	16.45 ± 12.96	**91.30 ± 12.29**	80.89 ± 9.00	81.22 ± 13.00	43.86 ± 9.64	79.89 ± 2.52
Pen-e	−2.41 ± 0.44	98.95 ± 47.69	127.33 ± 19.12	137.67 ± 8.27	**151.62 ± 4.92**	121.21 ± 11.09	133.29 ± 6.99
Pen-h	7.24 ± 5.99	−2.95 ± 1.85	**108.11 ± 17.62**	90.87 ± 20.47	81.58 ± 24.36	59.04 ± 8.19	94.03 ± 16.08
Door-c	−0.34 ± 0.00	−0.34 ± 0.00	**8.88 ± 2.60**	−0.08 ± 0.02	−0.06 ± 0.04	−0.06 ± 0.02	0.02 ± 0.03
Door-e	84.75 ± 11.70	−0.34 ± 0.00	105.44 ± 0.11	104.54 ± 1.82	104.81 ± 0.18	102.80 ± 5.65	**105.74 ± 0.75**
Door-h	1.28 ± 1.28	−0.27 ± 0.16	6.14 ± 1.31	3.69 ± 1.37	−0.11 ± 0.11	**6.69 ± 3.12**	0.76 ± 0.55
Relocate-c	−0.31 ± 0.01	−0.31 ± 0.01	0.09 ± 0.07	−0.04 ± 0.05	**1.03 ± 1.61**	−0.11 ± 0.04	0.02 ± 0.07
Relocate-e	−0.31 ± 0.02	−2.37 ± 0.41	**108.61 ± 1.21**	102.71 ± 1.71	103.49 ± 5.46	102.80 ± 4.35	107.51 ± 0.75
Relocate-h	0.02 ± 0.03	−0.30 ± 0.02	0.07 ± 0.03	**0.10 ± 0.04**	−0.03 ± 0.03	0.03 ± 0.02	0.09 ± 0.06

Table 2 (continued)

Task environment	CQL	TD3+BC	DT	IQL	DTQL	O-DICE	TACCO (Ours)
Hammer-c	0.30 ± 0.01	0.31 ± 0.11	1.34 ± 0.15	3.72 ± 3.05	**12.35 ± 9.76**	1.86 ± 0.85	4.82 ± 5.34
Hammer-e	0.15 ± 0.05	2.63 ± 0.94	99.23 ± 14.55	127.98 ± 0.31	**130.08 ± 0.34**	99.91 ± 16.43	130.03 ± 0.77
Hammer-h	0.13 ± 0.07	0.88 ± 0.26	1.71 ± 0.65	1.53 ± 1.53	−0.06 ± 0.28	0.83 ± 0.17	**3.92 ± 1.79**
Adroit total	103.00	114.64	658.25	553.58	566.02	538.86	**660.12**
Runtime (Hopper-e)	16 h 35 m 44 s	**3 h 16 m 08 s**	9 h 46 m 34 s	5 h 09 m 20 s	11 h 52 m 05 s	8 h 35 m 39 s	3 h 39 m 10 s
Runtime (Pen-c)	15 h 27 m 20 s	**2 h 49 m 01 s**	7 h 13 m 01 s	4 h 30 m 32 s	9 h 38 m 51 s	9 h 44 m 36 s	3 h 25 m 03 s
Rank-summary	first=0, second=4	first=1, second=1	first=5, second=4	first=3, second=1	first=5, second=4	first=2, second=1	**first=12, second=4**

Results are reported as mean ± 95% confidence interval over 5 random seeds. The optimal result in each environment is highlighted in bold. To assess relative strengths at a high level, we counted the total number of times TACCO achieved first and second place across all environments. Runtime denotes the training time required to complete one million environment steps. The suffixes in the environment names denote different dataset types:`-m-r` = medium-replay,`-m` = medium,`-m-e` = medium-expert,`-e` = expert,`-r` = random,`-c` = cloned, and`-h` = human

Within the high-Q candidate set, we further rank samples by uncertainty in ascending order and select the bottom ρ_u% samples as the final positive samples. The remaining samples are treated as negative samples.

The optimization objective of TACCO-B remains the same as that of the standard TD3+BC algorithm. After updating the critic parameters, we further introduce the contrastive learning mechanism and update the shared state–action encoder according to Eq. (6). The complete algorithmic procedure is described in greater detail in Appendix A.

4 Experiment

We rigorously tested the proposed algorithm on the D4RL benchmark, covering both Mujoco and Adroit environments to evaluate its performance in continuous control tasks with dense and sparse rewards. All experiments were conducted on a high-performance computing platform equipped with four NVIDIA RTX A5000 GPUs, one Intel i9-10920X CPU, and 256 GB of RAM. Algorithm performance was measured across two dimensions: normalized scores as the core metric and computational efficiency assessed by comparing time consumption at identical test step sizes. Furthermore, we examine the effectiveness of our proposed method in mitigating OOD action interference by analyzing the distance between positive and negative pairs and the critic disagreement. Finally, ablation experiments further validate the critical role of contrastive learning in improving overall performance.

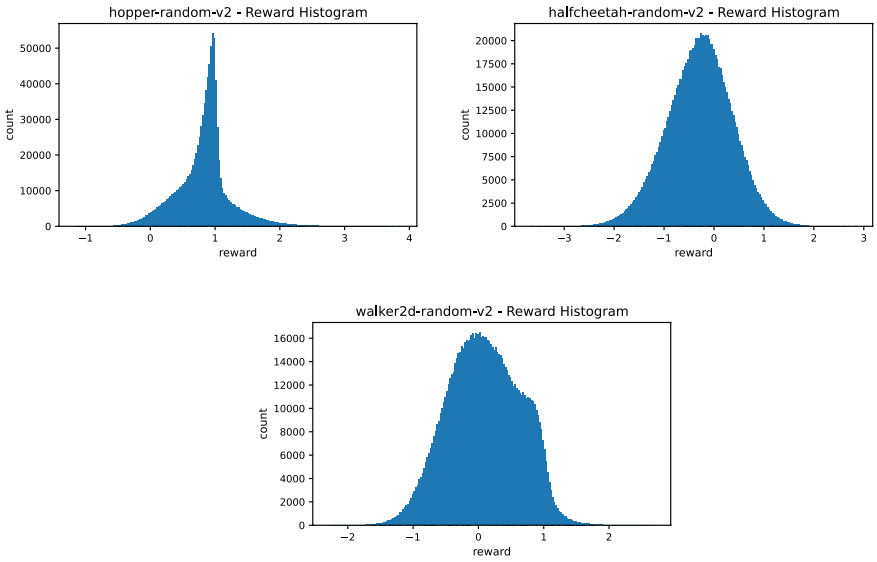


Fig. 3 Reward distribution histograms of three Mujoco random datasets

4.1 Results of TACCO on All Environments of the D4RL Benchmark

In this section, we present the experimental results of the TACCO algorithm on the Mujoco and Adroit environments from the D4RL benchmark. Specifically:

- **Mujoco environment**: The Mujoco environment is a classical benchmark in offline deep reinforcement learning, characterized by dense rewards and suboptimal policy datasets that are widely used to evaluate algorithmic performance. The datasets are categorized by quality in the following order: expert, medium-expert, medium, medium-replay, and random.
- **Adroit environment**: The Adroit environment represents a more challenging setting, consisting of high-dimensional robotic manipulation tasks with sparse rewards and narrow data distributions. It is primarily used to assess the effectiveness of algorithms in complex, real-world scenarios. The datasets are ordered by quality as: expert, human, and cloned.

We compare TACCO against several state-of-the-art baselines, including CQL Kumar et al. 2020, IQL Kostrikov et al. 2022, DT Chen et al. 2021, TD3+BC, DTQL Chen et al. 2024, and O-DICE Mao et al. 2024. All baseline methods were run under a uniform environment configuration, with algorithm hyperparameters primarily following the settings specified in the original papers; where not explicitly stated, the default parameters from the official implementations were used. The results reported in the Table 2 are normalized scores accompanied by 95% confidence intervals, calculated as the average of the last 10 evaluation results from the late training phase across five random seeds. The environment configuration is provided in Appendix A.3.

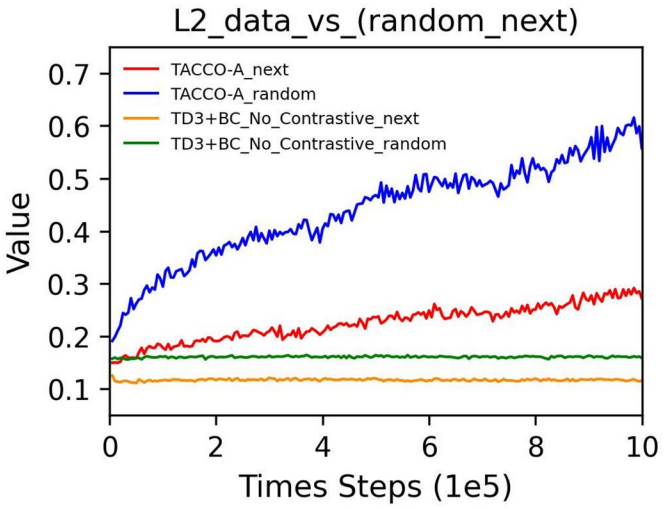


Fig. 4 The ℓ_2 distance between the action in the dataset and $\pi_\phi(s')$ and a_{random}

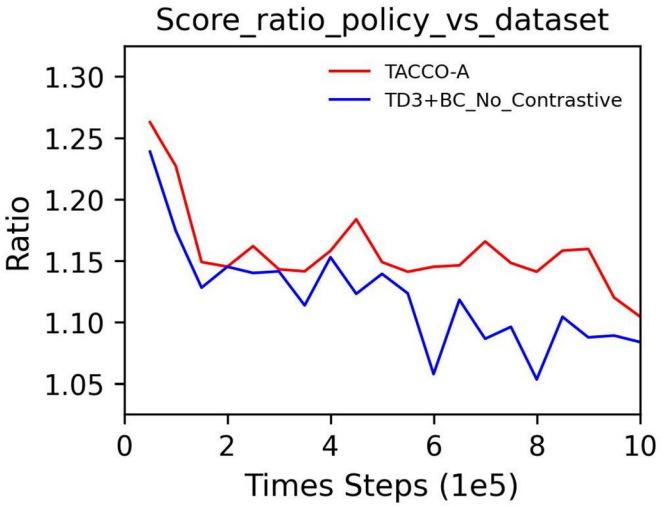


Fig. 5 Score ratio of policy to actions in dataset

In the Mujoco environment (except for walker2d-random), we employ the TACCO-A variant, while in the Adroit environment, we adopt TACCO-B. It is worth noting that although hopper-random, halfcheetah-random, and walker2d-random all share low data quality and limited effective experience, their learning difficulties are not identical. Specifically, Fig. 3 illustrates the single-step reward distributions in the hopper, halfcheetah, and walker2d-random environments. In hopper-random, due to its low action dimensionality, rewards are highly concentrated around 1, indicating that even random policies can frequently obtain positive feedback. In halfcheetah-random, the reward distribution peaks around −0.5 and exhibits a relatively symmetric shape, providing balanced positive and negative signals that enable

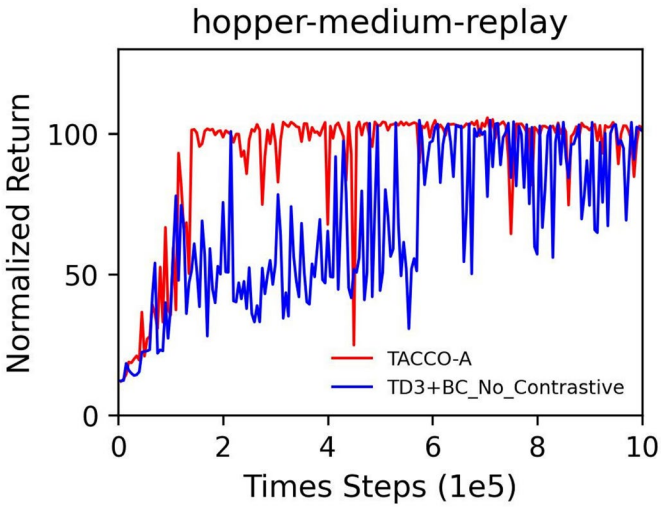


Fig. 6 The score of D4RL in hopper-medium-replay-v2 environment

effective learning. In contrast, in walker2d-random, the reward distribution peaks near 0 with smaller variance than in halfcheetah, suggesting that most random trajectories yield little meaningful feedback. Under such conditions, the policy network struggles to acquire stable learning signals, and therefore we apply the TACCO-B variant in this environment.

As shown in the results Table 2, across 27 evaluation tasks, TACCO outperforms other baselines, ranking first or second in 16 environments. Moreover, its computational cost is only slightly higher than that of TD3+BC, suggesting potential advantages for practical applications. Table 10 and Appendix A summarize the hyperparameter configurations used during training, Appendix B discusses the sensitivity of these hyperparameters and Fig. 11 in Appendix C shows the learning curve of the TACCO algorithm.

4.2 The Role of Contrastive Learning in TACCO

4.2.1 TACCO-A

We compare the TACCO-A algorithm with the TD3+BC algorithm without contrastive learning (defined as TD3+BC_No_Contrastive). We measure the ℓ_2 distance in the embedding space between in-distribution (ID) actions from the dataset and two types of negative samples, namely random actions and next-state policy actions $\pi_\phi(s')$. Specifically, for a given state s, we compute:

$$\ell_2(a, a^-) = \mathbb{E}_{(s,a)\sim\mathcal{D}, a^-\sim p^-(\cdot|s)}\left[\left\|e_\phi(s,a) - e_\phi(s,a^-)\right\|_2\right], \tag{14}$$

where a^- denotes either a random action (a_{random}) or a next-state policy action ($\pi_\phi(s')$) and $p^-(\cdot \mid s)$ denotes sampling an action from the negative sample distribution given state s. This metric reflects how well the learned representation separates in-support actions from negative samples.

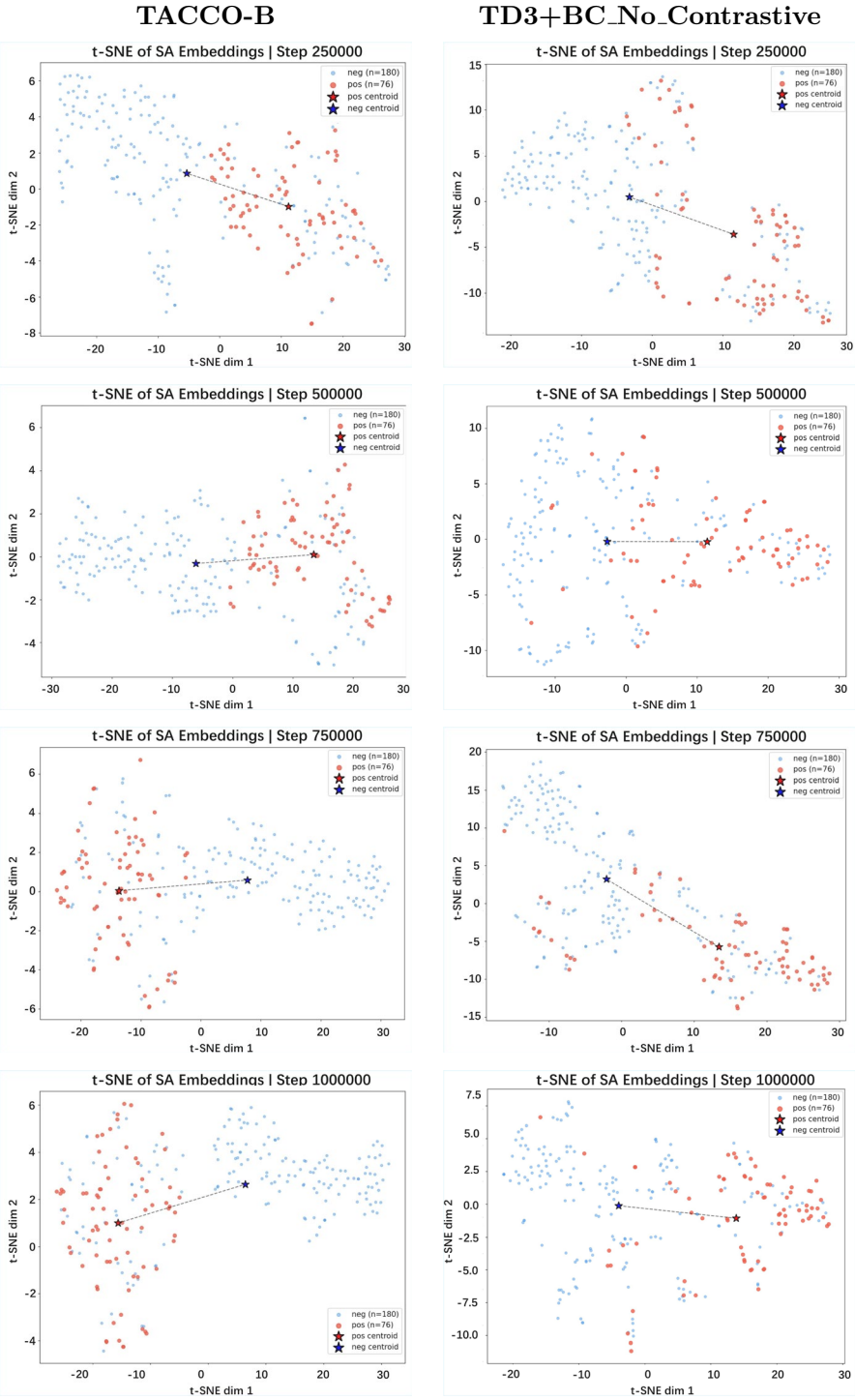


Fig. 7 t-SNE visualization of embeddings learned by TACCO-B (left) and TD3+BC_No_Contrastive (right) in the hammer-human-v1 environment

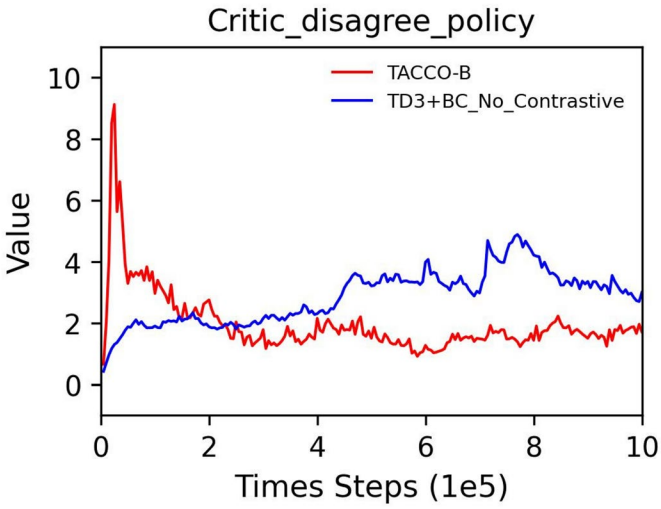


Fig. 8 Critic disagreement analysis results in the hammer-human-v1 environment

As shown in Fig. 4, in TACCO-A, the distance between ID actions and both types of negative samples increases steadily during training, whereas the baseline without contrastive learning remains nearly constant. This result directly demonstrates that the contrastive learning objective effectively separates data-supported actions from negative samples in the representation space, thereby forming a clearer positive–negative separation structure.

In addition, we report the nearest-neighbor distance ratio between policy actions and dataset actions, denoted as `score_ratio_policy_vs_dataset`, as a proxy for the extent to which the learned policy deviates from the data support. The computation follows the same procedure as in Eq. (8). As shown in Fig. 5, both methods exhibit comparable values on this metric, indicating that TACCO-A does not significantly push the learned policy away from the support of the dataset. The relatively small difference on this metric is consistent with the characteristics of the hopper-medium-replay dataset, which contains a mixture of random and medium-quality policies.

In addition, Fig. 6 shows the training curves for the two algorithms. The D4RL score curve for TACCO-A enters the high-score range earlier and maintains relatively more stable performance during subsequent training. This suggests that contrastive learning reduces the actor's tendency to generate unfamiliar or OOD actions by establishing a clearer separation between positive and negative pairs in the representation space, thereby improving the stability of the training process.

4.2.2 TACCO-B

For the TACCO-B algorithm, since positive and negative pairs do not represent different actions within the same state, it is not possible to use ℓ_2 distance to visualize the separation of positive and negative samples. Therefore, we used a t-SNE plot for visualization. As shown in Fig. 7, the TACCO-B on the left exhibits a clearer and more stable trend of separation between positive and negative samples compared to the TD3+BC_No_Contrastive on the right. While the baseline method without contrastive learning shows some separation in

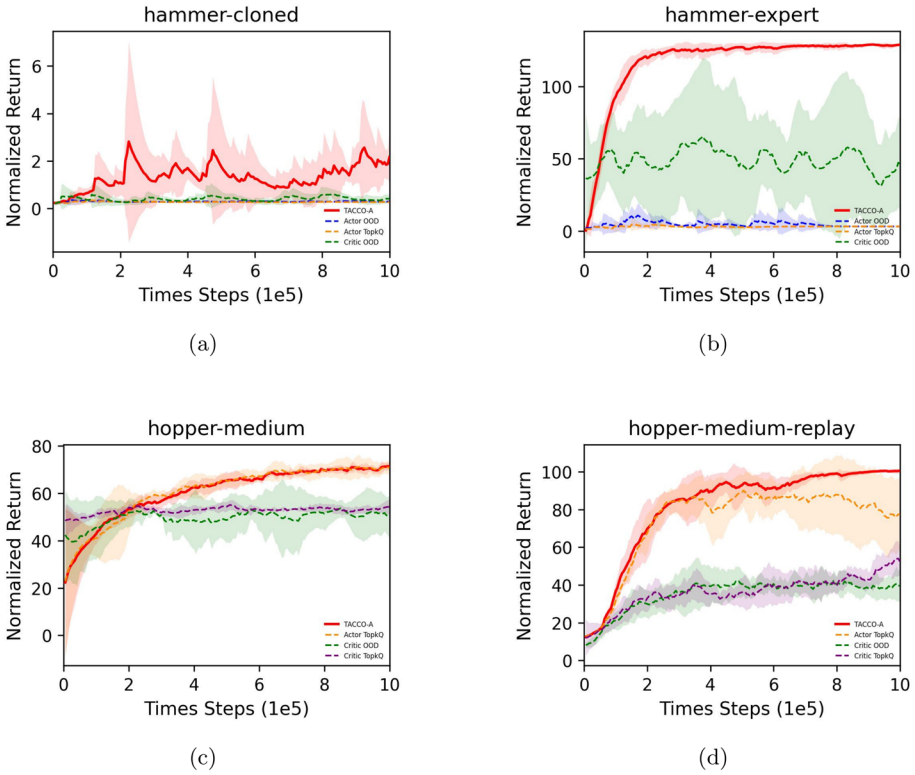


Fig. 9 Ablation experiment of contrastive learning in TACCO across actor and critic components, evaluated in Mujoco and Adroit environments. In this ablation experiment, we set four variables: Actor, Critic, TopkQ (state-action pairs with high Q-values and low uncertainty), and OOD ((s,a_{random}) and (s,$\pi_\phi(s')$)). We paired actor and critic with each of the two state-action categories and named the combinations accordingly (e.g., Actor TopkQ). TACCO denotes the optimal combination. To ensure fairness, unused parameters were fixed at 0.5

certain phases, the overall overlap is more pronounced, the structure is more dispersed, and the separation pattern is less stable.

Additionally, we have included the results of the critic disagreement assessment. We quantify critic disagreement on policy actions as the mean absolute difference between the two critic estimates evaluated at the current policy action:

$$\text{CriticDisagree}_{\text{policy}} = \mathbb{E}_{s\sim\mathcal{D}}\left[|Q_1(s,\pi(s)) - Q_2(s,\pi(s))|\right]. \tag{15}$$

As shown in Fig. 8, after introducing contrastive learning, the critic disagreement of the TACCO-B algorithm is significantly lower than that of TD3+BC_No_Contrastive and remains relatively stable throughout training; in contrast, the critic disagreement of TD3+BC_No_Contrastive remains at a higher level over the long term and exhibits more pronounced fluctuations. This indicates that contrastive learning helps ensure that the actions generated by the policy fall within a region where the critic's evaluation is more consistent and reliable, thereby reducing the uncertainty in value estimation.

Table 3 Performance comparison of TACCO_A and TACCO_B across D4RL benchmarks (mean ± 95% CI) over 5 seeds. BC weight α, contrastive weight β , Q-threshold ρ_q, uncertainty threshold ρ_u are all set to 0.5

Environment	TACCO_A mean ± 95% CI	TACCO_B mean ± 95% CI
Halfcheetah-random-v2	**15.54** ± 0.45	4.89 ± 1.61
Halfcheetah-medium-v2	**50.33** ± 0.12	45.98 ± 0.18
Halfcheetah-medium-replay-v2	**46.90** ± 0.56	42.16 ± 0.25
Halfcheetah-medium-expert-v2	79.25 ± 7.75	**93.52** ± 0.24
Halfcheetah-expert-v2	92.20 ± 3.21	**95.67** ± 0.41
Hopper-random-v2	13.98 ± 8.03	**17.94** ± 10.21
Hopper-medium-v2	**66.37** ± 4.46	56.27 ± 3.31
Hopper-medium-replay-v2	**79.35** ± 11.64	41.79 ± 11.96
Hopper-medium-expert-v2	**109.48** ± 2.04	86.58 ± 7.40
Hopper-expert-v2	**108.10** ± 4.20	106.40 ± 5.79
Walker2d-random-v2	2.36 ± 2.02	**11.44** ± 4.81
Walker2d-medium-v2	**85.37** ± 1.87	76.21 ± 3.90
Walker2d-medium-replay-v2	**89.25** ± 1.50	18.96 ± 13.44
Walker2d-medium-expert-v2	**110.92** ± 0.29	108.61 ± 0.59
Walker2d-expert-v2	90.50 ± 35.71	**109.54** ± 0.18
MuJoCo Total	**1039.95**	916.05
Door-human-v1	−0.05 ± 0.02	**0.04** ± 0.04
Door-cloned-v1	−0.34 ± 0.00	**−0.03** ± 0.04
Door-expert-v1	104.24 ± 1.35	**104.31** ± 1.58
Hammer-human-v1	**3.49** ± 1.67	1.73 ± 0.73
Hammer-cloned-v1	0.25 ± 0.01	**2.03** ± 2.24
Hammer-expert-v1	3.14 ± 0.01	**99.35** ± 42.52
Relocate-human-v1	**0.07** ± 0.07	0.00 ± 0.04
Relocate-cloned-v1	−0.31 ± 0.00	**−0.16** ± 0.04
Relocate-expert-v1	104.06 ± 3.29	**107.19** ± 0.64
Pen-human-v1	67.83 ± 2.61	**94.04** ± 20.18
Pen-cloned-v1	57.68 ± 11.09	**79.89** ± 2.52
Pen-expert-v1	**128.49** ± 8.07	127.09 ± 3.92
Adroit total	368.64	**615.48**

4.3 Ablation Experiment of Contrastive Learning in TACCO

4.3.1 Contrastive Learning on Actor and Critic in TACCO

When designing contrastive learning, we set different contrastive pairs for the actor and critic. In dense reward environments, we treated state-action pairs from D4RL as positive examples and OOD-like state-action pairs as negative examples. In sparse reward environments, we selected a proportion of high-Q, low-uncertainty state-action pairs as positive examples, with the remainder serving as negative examples. As shown in Fig. 9, the results indicate that in a dense reward environment, applying contrastive learning to the actuator and directly using OOD action constraints more effectively guides the generated policy away from out-of-distribution actions, thereby achieving higher scores. Conversely, reward-sparse environments rely more heavily on the accuracy of Q-value estimates to ensure the reliability of the generated policy, making it more appropriate to apply contrastive learning to the critic. In addition, we trained both the TACCO-A and TACCO-B algorithms across all environments with all hyperparameters fixed at 0.5, and the experimental results in Table 3 similarly support the above conclusion.

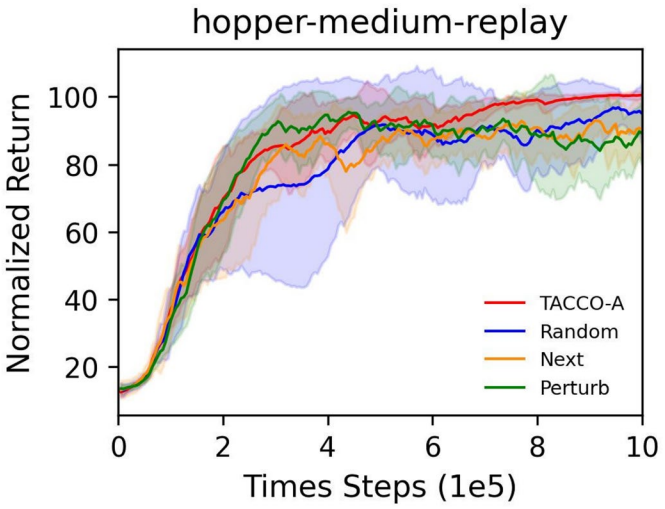


Fig. 10 Ablation study on negative sample selection

4.3.2 Selection of Negative Sample in TACCO-A

At the same time, we conducted ablation experiments on the negative samples for TACCO-A. To validate the rationality of the selected negative samples, we compared the impact of four negative sample strategies on training in the hopper-medium-replay environment: using only random actions (Random), using only actions from the next-state policy (Next), using actions from the perturbed dataset (Perturb), and the combined approach proposed in this paper (TACCO-A).

As shown in Fig. 10, the three single-negative-sample strategies (Random, Next, and Perturb) achieved similar final performance, as they essentially share the same objective: to construct OOD-like state-action pairs that deviate from the support region of the dataset distribution, thereby providing effective rejection signals for contrastive learning. Nevertheless, TACCO-A achieved further performance improvements over these three strategies. We believe this stems from the complementary coverage of different types of negative samples in the OOD space: random actions provide a broad region far from the distribution in the action space, while the next-state policy actions provide challenging negative samples close to the distribution boundary. The combination of the two makes the contrastive constraint more comprehensive, leading to more stable and superior policy performance.

4.4 Ablation Experiments with Consistent Parameters

To objectively evaluate the gain effect of contrastive learning, we fully aligned TD3+BC with TACCO in terms of hyperparameters and training configurations, and denoted the former as TD3+BC_Weight. Experimental results show that TACCO maintains overall superiority across 27 environments, significantly outperforming TD3+BC_Weight. This demonstrates that contrastive learning effectively suppresses the policy's preference for OOD actions, compelling it to learn within the in-distribution action space. Consequently, the policy achieves more stable and higher returns. Table 4 presents the detailed experimental results.

Table 4 Comparison of TD3+BC_Weight and TACCO (Ours) on Mujoco and Adroit tasks

Task environment	TD3+BC_Weight	TACCO (Ours)	TACCO vs TD3+BC_Weight (%)
Halfcheetah-medium-replay	52.04 ± 1.63	**52.36 ± 0.98**	+0.6%
Halfcheetah-medium	**58.96 ± 0.44**	58.34 ± 0.42	−1.1%
Halfcheetah-medium-expert	90.99 ± 2.78	**94.13 ± 1.64**	+3.5%
Halfcheetah-expert	97.23 ± 0.56	**97.77 ± 0.42**	+0.5%
Halfcheetah-random	**29.58 ± 1.47**	29.39 ± 1.48	−0.7%
Hopper-medium-replay	65.58 ± 21.32	**101.31 ± 0.86**	+54.5%
Hopper-medium	68.90 ± 4.85	**70.52 ± 5.04**	+2.4%
Hopper-medium-expert	106.85 ± 4.41	**109.48 ± 2.04**	+2.5%
Hopper-expert	111.86 ± 0.37	**112.29 ± 0.43**	+0.4%
Hopper-random	20.90 ± 11.55	**31.70 ± 0.43**	+51.6%
Walker2d-medium-replay	86.04 ± 2.67	**89.25 ± 1.50**	+3.7%
Walker2d-medium	83.23 ± 3.00	**85.07 ± 2.14**	+2.2%
Walker2d-medium-expert	91.53 ± 20.69	**111.61 ± 0.24**	+21.9%
Walker2d-expert	91.75 ± 33.76	**111.84 ± 0.44**	+21.9%
Walker2d-random	4.93 ± 1.14	**12.77 ± 6.14**	+159.2%
Pen-cloned	71.41 ± 15.38	**79.89 ± 2.52**	+11.9%
Pen-expert	131.39 ± 3.78	**133.29 ± 6.99**	+1.4%
Pen-human	69.30 ± 24.47	**94.03 ± 16.08**	+35.7%
Door-cloned	−0.07 ± 0.04	**0.02 ± 0.03**	+128.9%
Door-expert	104.91 ± 0.13	**105.74 ± 0.75**	+0.8%
Door-human	**1.13 ± 1.35**	0.76 ± 0.55	−32.8%
Relocate-cloned	−0.04 ± 0.03	**0.02 ± 0.07**	+148.9%
Relocate-expert	106.23 ± 1.38	**107.51 ± 0.75**	+1.2%
Relocate-human	0.04 ± 0.03	**0.09 ± 0.06**	+148.8%
Hammer-cloned	0.32 ± 0.14	**4.82 ± 5.34**	+1419.9%
Hammer-expert	2.65 ± 0.97	**130.03 ± 0.77**	+4814.5%
Hammer-human	2.42 ± 2.23	**3.92 ± 1.79**	+62.2%
Rank Summary	first=3, second=24	first=24, second=3	–

Experimental settings are consistent with Sect. 4.1. The final column reports the performance improvement of TACCO over TD3+BC_Weight (percentage)

5 Conclusion

This paper proposed an offline RL algorithm named TACCO based on TD3+BC. By distinguishing between OOD-like state-action pairs and in-distribution state-action pairs in the representation space, the algorithm effectively alleviates the OOD action interference problem and enhances policy stability. Unlike traditional approaches relying on policy constraints or value regularization, TACCO adopts a representation learning perspective, enabling the actor and critic to leverage their respective strengths in dense and sparse reward environments, thereby achieving more stable and efficient policy updates. Experiments on the D4RL benchmark across Mujoco and Adroit environments demonstrate that TACCO significantly outperforms existing state-of-the-art algorithms in most tasks. It ranks among the top two in 16 out of 27 environments while maintaining computational efficiency comparable to TD3+BC. These results indicate that contrastive learning offers a practical new approach to addressing OOD challenges in offline RL.

Appendix A: Network Architecture and Algorithmic Procedure

TACCO-A

TACCO-A is built upon TD3+BC and introduces a supervised contrastive learning branch based on state–action pairs on the actor side. In the current implementation, the contrastive learning module operates only on the actor, not on the critic. The actor consists of a shared state-encoding backbone, a policy output head, and an embedding head for contrastive learning.

Given a state $s \in \mathbb{R}^{d_s}$ (d_a denotes the dimension of the action and d_s denotes the dimension of the state.), a hidden representation $h(s)$ is first obtained via a two-layer multilayer perceptron (MLP):

$$h(s) = g_\phi(s),$$

where g_ϕ maps:

$$d_s \to 256 \to 256,$$

with a ReLU activation applied after each linear transformation. Subsequently:

- The **policy output head** is identical to that of TD3+BC; it takes the hidden state representation $h(s)$ and outputs an action.
- The **contrastive learning embedding head** concatenates $h(s)$ and the action a, then projects the result into a 64-dimensional representation:

$$z(s, a) = e_\phi([h(s),\, a]),$$

where

$$e_\phi \;:\; (256 + d_a) \to 256 \to 64,$$

defines the layer counts and neuron sizes of the contrastive learning module. A ReLU activation is applied after the intermediate layer. The resulting 64-dimensional output is the representation vector used for contrastive learning in this work.The critic adopts the dual Q-network structure from TD3, denoted Q_{ψ_1} and Q_{ψ_2}. Each Q-network takes the concatenated state–action vector $[s, a]$ as input and is implemented as a three-layer MLP:

$$(d_s + d_a) \to 256 \to 256 \to 1,$$

with ReLU activations in the hidden layers. The two Q-networks are independent and share no parameters.

In TACCO-A, the shared layer is the actor encoder g_ϕ. No normalization scheme is applied to it. The specific parameters in Table 5 are as follows:

Table 5 TACCO-A hyperparameters

Hyperparameter	Value
γ	0.99
τ	0.005
policy_noise	0.2
noise_clip	0.5
policy_freq	2
batch_size	256
max_timesteps	10^6
actor learning rate	3×10^{-4}
critic learning rate	3×10^{-4}
embedding dimension	64
temperature	0.07
normalize	True
normalize_reward	False

TACCO-B

TACCO-B is likewise built upon TD3+BC, but introduces supervised contrastive learning at the state–action representation layer of the critic. Unlike TACCO-A, the contrastive learning branch in TACCO-B is not attached to the actor; instead, it is attached to a state–action encoder internal to the critic.

Actor: The actor is a standard TD3-BC policy network. Given state $s \in \mathbb{R}^{d_s}$, the policy network is a three-layer MLP:

$$d_s \to 256 \to 256 \to d_a,$$

where ReLU activations follow the first two layers, and the final layer is followed by tanh multiplied by the action range `max_action`, producing a continuous action.

Critic: The critic is composed of a shared state–action encoder and two independent Q-value heads. Given a state–action pair (s, a), the inputs are first concatenated into $[s, a]$ and fed into the encoder:

$$z(s, a) = f_\psi([s, a]).$$

The encoder has the following structure:

$$(d_s + d_a) \to 64 \to 64 \to 64,$$

with ReLU activations after the first two layers, followed by a LayerNorm at the output. Thus the encoder produces a 64-dimensional contrastive state–action embedding.

The embedding $z(s, a)$ is then passed to two independent Q-value heads:

$$Q_{\theta_1}(s, a) = h_1(z(s, a)), \qquad Q_{\theta_2}(s, a) = h_2(z(s, a)).$$

Each Q-head has the structure:

$$64 \to 256 \to 256 \to 1,$$

with ReLU activations in the hidden layers. The two Q-heads are independent but share the same contrastive state–action encoder. In TACCO-B, the shared layer is the critic state–action encoder f_ψ, to which the contrastive learning objective is applied. LayerNorm is used at the encoder output. The specific parameters in Table 6 are as follows:

Table 6 TACCO-B hyperparameters

Hyperparameter	Value
γ	0.99
τ	0.005
δ	0.01
policy_noise	0.2
noise_clip	0.5
policy_freq	2
batch_size	256
max_timesteps	10^6
actor learning rate	3×10^{-4}
critic learning rate	3×10^{-4}
encoder learning rate	1×10^{-4}
embedding dimension	64
temperature	0.07
normalize	True
normalize_reward	True

Evaluation Protocol and Baseline Network Details

All experiments, including all baseline algorithms (CQL, TD3+BC, DT, IQL, DTQL, O-DICE) and TACCO, were conducted under the following identical protocol:

- **Environment versions:** MuJoCo (`-v2`), Adroit (`-v1`), following the standard D4RL benchmark Fu et al. 2020.
- **Score normalization:** D4RL standard normalization, computed as $\text{normalized score} = 100 \times \frac{\text{score} - \text{random score}}{\text{expert score} - \text{random score}}$.
- **Training steps:** 1×10^6 gradient steps for all methods.
- **Evaluation frequency:** Every 5, 000 training steps for all methods.
- **Number of seeds:** 5 random seeds for all methods.
- **Batch size:** 256 for all methods (64 for DT).
- **Results reporting:** Mean ± 95% confidence interval over 5 random seeds, computed as $\bar{x} \pm 1.96 \times \sigma/\sqrt{5}$.

Network sizes for each baseline are summarized in Table 7.

Table 7 Network sizes and batch sizes of baseline algorithms

Algorithm	Actor/Policy	Critic/Value	Batch Size
CQL	MLP ×3 (256)	Double Q, MLP ×3 (256)	256
TD3+BC	MLP ×2 (256)	Double Q, MLP ×2 (256)	256
IQL	MLP ×2 (256)	Double Q + Value net, MLP ×2 (256)	256
O-DICE	MLP ×2 (1024)	Double V, MLP ×2 (256)	256
DT	Transformer (embedding=128, heads=1, layers=3)	—	64

All MLP-based methods use ReLU activations. Double Q denotes a double Q-network critic; Value net denotes an additional state-value network in IQL; Double V denotes a double value network in O-DICE. DT uses a causal Transformer with sequence length 20

Appendix B: Hyperparameter Sensitivity Analysis

We conduct sensitivity analyses on two sets of key hyperparameters.

TACCO-A: `BC_Weight (`$\boldsymbol{\alpha}$`)` **and** `Supcon_Weight (`$\boldsymbol{\beta}$`).`

We analyze the effects of α and β on the hopper-medium-replay-v2 task. A smaller α relaxes the behavioral cloning constraint, allowing the policy to venture beyond the support of the offline dataset and explore potentially high-reward regions; however, this also introduces the risk of training instability, as the policy may frequently visit OOD state-action pairs, leading to increased error in value function estimation. On the other hand, a larger β encourages the critic to learn more structured and discriminative representations, substantially improving training stability. Experimental results in Table 8 indicate that performance is more sensitive to α, whereas it remains relatively robust to variations in β. Notably, as β increases, mean performance improves steadily, suggesting that contrastive learning can effectively compensate for the instability introduced by a weaker behavioral cloning constraint.

TACCO-B: `Percent_q (`$\boldsymbol{\rho_q}$`)` **and** `Percent_u (`$\boldsymbol{\rho_u}$`).`

We conduct sensitivity experiments on the hyperparameters ρ_q and ρ_u on the hammer-human-v1 task to analyze their effects on performance. Experimental results in Table 9 indicate that the model is relatively sensitive to ρ_q. Specifically, when ρ_q is small, the model relies on only a limited number of high-value samples, resulting in insufficient data coverage and degraded performance stability; conversely, when ρ_q is too large, more low-quality or OOD actions are introduced, which in turn increases training variance. In contrast, the model exhibits lower sensitivity to ρ_u. Although extreme values—either too small or too large—lead to performance degradation due to limited samples or excessive uncertainty, respectively, the model maintains relatively stable performance across moderate to large values.

Table 8 Sensitivity analysis of α and β on hopper-medium-replay-v2 (mean ± 95% CI over 5 seeds)

α	β	mean ± 95% CI
Varying β (bc_weight = 0.1)		
0.1	0.0	78.09 ± 24.06
0.1	0.1	85.93 ± 15.68
0.1	0.2	70.87 ± 14.65
0.1	0.3	99.50 ± 5.82
0.1	0.4	96.67 ± 4.13
0.1	0.5	94.37 ± 12.13
0.1	0.6	88.95 ± 14.41
0.1	0.7	76.86 ± 18.56
0.1	0.8	99.72 ± 4.73
0.1	0.9	101.31 ± 0.85
0.1	1.0	**101.33** ± 1.08
Varying α (supcon_weight = 0.0)		
0.1	0.0	78.09 ± 24.06
0.2	0.0	**86.92** ± 19.85
0.3	0.0	75.29 ± 25.17
0.4	0.0	78.46 ± 14.43
0.5	0.0	86.74 ± 13.99
0.6	0.0	76.32 ± 19.97
0.7	0.0	84.31 ± 14.77
0.8	0.0	81.10 ± 8.02
0.9	0.0	69.34 ± 16.78
1.0	0.0	68.51 ± 20.21

Table 9 Sensitivity analysis of ρ_q and ρ_u on hammer-human-v1 (mean ± 95% CI over 5 seeds)

ρ_q	ρ_u	mean ± 95% CI
Varyingρ_q (ρ_u = 1.0)		
0.1	1.0	3.14 ± 3.50
0.2	1.0	1.73 ± 0.95
0.3	1.0	2.07 ± 1.30
0.4	1.0	0.94 ± 0.23
0.5	1.0	0.93 ± 0.26
0.6	1.0	1.29 ± 0.10
0.7	1.0	1.40 ± 0.82
0.8	1.0	0.84 ± 0.45
0.9	1.0	**3.67** ± 3.28
1.0	1.0	2.74 ± 2.42
Varyingρ_u (ρ_q = 0.5)		
0.5	0.1	2.61 ± 2.71
0.5	0.2	1.63 ± 0.69
0.5	0.3	2.46 ± 3.02
0.5	0.4	0.93 ± 0.26
0.5	0.5	1.73 ± 0.73
0.5	0.6	**3.92** ± 1.79
0.5	0.7	1.05 ± 0.17
0.5	0.8	1.09 ± 0.34
0.5	0.9	0.68 ± 0.08

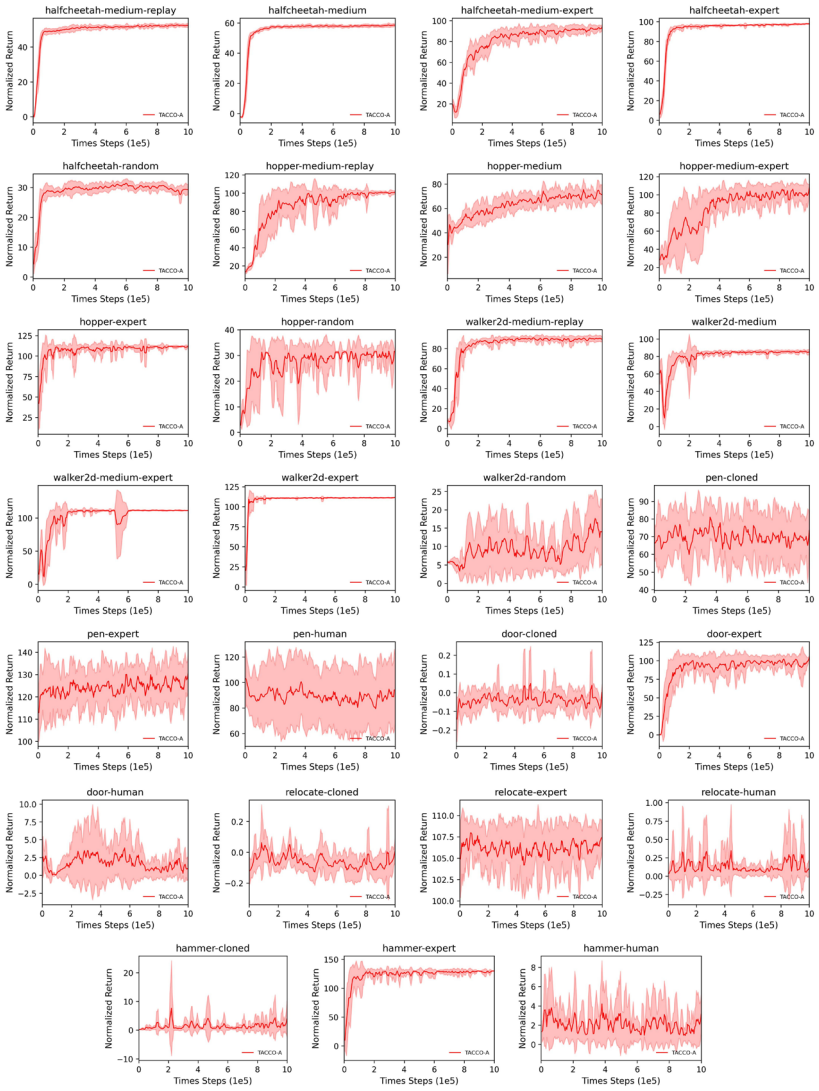


Fig. 11 Learning curves of TACCO (Ours) across 27 Mujoco and Adroit environments

Appendix C: Learning Curves and Parameter Settings of TACCO

Table 10 Hyperparameter settings for Mujoco and Adroit environments

Task environment	α	β	η
TACCO-A			
HalfCheetah-m	0.1	0.9	2.5
HalfCheetah-e	0.9	0.1	2.5
HalfCheetah-m-e	0.9	0.1	2.5
HalfCheetah-m-r	0.1	0.9	2.5
HalfCheetah-r	0	1	2.5
Hopper-m	0.4	0.2	2.5
Hopper-e	0.9	0.1	2.5
Hopper-m-e	0.5	0.5	2.5
Hopper-m-r	0.1	0.9	2.5
Hopper-r	0	1	2.5
Walker2d-m	0.7	0.3	2.5
Walker2d-e	0.5	1	2.5
Walker2d-m-e	0.5	0.6	2.5
Walker2d-m-r	0.5	0.5	2.5
Task environment	ρ_q	ρ_u	η
TACCO-B			
Door-cloned	0.2	0.1	1
Door-expert	0.3	0.3	0.01
Door-human	0.6	0.4	0.01
Hammer-cloned	0.2	0.1	1
Hammer-expert	0.1	0.3	1
Hammer-human	0.5	0.6	0.01
Pen-cloned	0.5	0.5	0.01
Pen-expert	0.3	0.3	0.01
Pen-human	0.2	0.5	0.01
Relocate-cloned	0.1	0.5	1
Relocate-expert	0.1	0.3	0.01
Relocate-human	0.1	0.7	0.01
Walker2d-r	0.2	0.2	1

The task environment naming is consistent with Table 2

Author Contributions T.H.Z. contributed to methodology, conducted the formal analysis and prepared the original draft. T.H.Z. and C.W. contributed to conceptualization. T.H.Z., C.W., and J.M. reviewed and edited the manuscript. J.M. acquired funding, provided resources, and supervised the project. All authors reviewed the manuscript.

Funding This work was supported in part by the Korea Institute of Energy Technology Evaluation and Planning(KETEP), the Ministry of Trade, Industry & Energy(MOTIE) of the Republic of Korea (RS-2024-00422103), in part by the Institute of Information & communications Technology Planning and Evaluation (IITP) grant funded by the Korea government (Ministry of Science and ICT, MSIT) (RS-2020-II201373), and in part by the Korea Institute of Energy Technology Evaluation and Planning(KETEP) and the Ministry of Climate, Energy & Environment(MCEE) of the Republic of Korea (NO. RS-2026-25530097).

Data Availability No datasets were generated or analysed during the current study.

Materials Availability Not applicable.

Code Availability Code will be made available on request.

Declarations

Conflict of interest The authors have no conflict of interest to declare that are relevant to the content of this article.

Ethical Approval and Consent to Participate The authors give their consent to participate.

Consent for Publication The authors give their consent for publication.

References

Afsar, M. M., Crump, T., & Far, B. (2022). Reinforcement learning based recommender systems: A survey. *ACM Computing Surveys, 55*(7), 1–38.

An, G., Moon, S., Kim, J.-H., & Song, H. O. (2021). Uncertainty-based offline reinforcement learning with diversified q-ensemble. *Advances in neural information processing systems, 34*, 7436–7447.

Baevski, A., Zhou, Y., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. *Advances in Neural Information Processing Systems, 33*, 12449–12460.

Bai, C., Wang, L., Yang, Z., Deng, Z., Garg, A., Liu, P., & Wang, Z. (2022) Pessimistic bootstrapping for uncertainty-driven offline reinforcement learning. arXiv preprint arXiv:2202.11566

Beeson, A., & Montana, G. (2022). Improving TD3-BC: Relaxed Policy Constraint for Offline Learning and Stable Online Fine-Tuning arXiv:2211.11802.

Beeson, A., & Montana, G. (2024). Balancing policy constraint and ensemble size in uncertainty-based offline reinforcement learning. *Machine Learning, 113*(1), 443–488.

Bengio, Y. (2013). Deep Learning of Representations: Looking Forward. *International Conference on Statistical Language and Speech Processing* (pp. 1–37). Springer.

Bengio, Y., Courville, A., & Vincent, P. (2013). Representation Learning: A Review and New Perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 35*(8), 1798–1828.

Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., & Mordatch, I. (2021). Decision Transformer: Reinforcement Learning via Sequence Modeling. *Advances in Neural Information Processing Systems, 34*, 15084–15097.

Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A Simple Framework for Contrastive Learning of Visual Representations. *International Conference on Machine Learning* (pp. 1597–1607) PMLR.

Chen, T., Wang, Z., & Zhou, M. (2024). Diffusion Policies Creating a Trust Region for Offline Reinforcement Learning. *Advances in Neural Information Processing Systems, 37*, 50098–50125.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 4171–4186)

Dulac-Arnold, G., Mankowitz, D., & Hester, T. (2019). Challenges of Real-World Reinforcement Learning arXiv:1904.12901.

Eysenbach, B., Zhang, T., Levine, S., & Salakhutdinov, R. R. (2022). Contrastive Learning as Goal-Conditioned Reinforcement Learning. *Advances in Neural Information Processing Systems, 35*, 35603–35620.

Fujimoto, S., & Gu, S. S. (2021). A Minimalist Approach to Offline Reinforcement Learning. *Advances in Neural Information Processing Systems, 34*, 20132–20145.

Fujimoto, S., Hoof, H., & Meger, D. (2018) Addressing function approximation error in actor-critic methods: S. Fujimoto et al. Proceedings of the 35th International Conference on Machine Learning (ICML) 80, 1587–1596

Fujimoto, S., Meger, D., & Precup, D. (2019). Off-Policy Deep Reinforcement Learning without Exploration. *International Conference on Machine Learning* (pp. 2052–2062) PMLR.

Fu, J., Kumar, A., Nachum, O., Tucker, G., & Levine, S. (2020). D4rl: Datasets for deep data-driven reinforcement learning arXiv:2004.07219 arXiv preprint.

Geng, Y., Ji, J., Chen, Y., Geng, H., Zhong, F., & Yang, Y. (2025) ReDMan: reliable dexterous manipulation with safe reinforcement learning: Y. Geng et al. Machine Learning 114(8), 189

Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P., Buchatskaya, E., Doersch, C., Avila Pires, B., Guo, Z., Gheshlaghi Azar, M., et al. (2020). Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning. *Advances in Neural Information Processing Systems, 33*, 21271–21284.

Han, S., & Sung, Y. (2021). Diversity Actor-Critic: Sample-Aware Entropy Regularization for Sample-Efficient Exploration. *International Conference on Machine Learning* (pp. 4018–4029) PMLR.

Hare, J. (2019). Dealing with Sparse Rewards in Reinforcement Learning arXiv:1910.09281.

He, K., Fan, H., Wu, Y., Xie, S., & Girshick, R. (2020). Momentum Contrast for Unsupervised Visual Representation Learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 9729–9738)

Hu, H., Wang, X., Zhang, Y., Chen, Q., & Guan, Q. (2024). A Comprehensive Survey on Contrastive Learning. *Neurocomputing, 610*, Article 128645.

Khosla, P., Teterwak, P., Wang, C., Sarna, A., Tian, Y., Isola, P., Maschinot, A., Liu, C., & Krishnan, D. (2020). Supervised Contrastive Learning. *Advances in Neural Information Processing Systems, 33*, 18661–18673.

Kostrikov, I., Nair, A., & Levine, S. (2022). Offline Reinforcement Learning with Implicit Q-Learning. International Conference on Learning Representations

Kumar, A., Zhou, A., Tucker, G., & Levine, S. (2020). Conservative Q-Learning for Offline Reinforcement Learning. *Advances in Neural Information Processing Systems, 33*, 1179–1191.

Laskin, M., Srinivas, A., & Abbeel, P. (2020). CURL: Contrastive Unsupervised Representations for Reinforcement Learning. *International Conference on Machine Learning* (pp. 5639–5650) PMLR.

Le-Khac, P. H., Healy, G., & Smeaton, A. F. (2020). Contrastive Representation Learning: A Framework and Review. *IEEE Access, 8*, 193907–193934.

Levine, S., Kumar, A., Tucker, G., & Fu, J. (2020). Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems arXiv:2005.01643.

Li, Y. (2017). Deep Reinforcement Learning: An Overview arXiv:1701.07274.

Liu, Z., Fang, Z., Lyu, J., & Li, X. (2025). Leveraging Score-Based Models for Generating Penalization in Model-Based Offline Reinforcement Learning. *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems* (pp. 1389–1398)

Lyu, J., Ma, X., Li, X., & Lu, Z. (2022). Mildly conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems, 35*, 1711–1724.

Ma, Y., Hao, J., Hu, X., Zheng, Y., & Xiao, C. (2024). Iteratively Refined Behavior Regularization for Offline Reinforcement Learning. *Advances in Neural Information Processing Systems, 37*, 56215–56243.

Mao, L., Xu, H., Zhang, W., & Zhan, X. (2024). ODICE: Revealing the Mystery of Distribution Correction Estimation via Orthogonal-Gradient Update arXiv:2402.00348.

Matheron, G., Perrin, N., & Sigaud, O. (2019). The Problem with DDPG: Understanding Failures in Deterministic Environments with Sparse Rewards arXiv:1911.11679.

Remmerden, J., Bukhsh, Z., & Zhang, Y. (2025). Offline reinforcement learning for learning to dispatch for job shop scheduling. *Machine Learning, 114*(8), 191.

Shan, Y., Zhu, Z., Long, T., Liang, Q., Chang, Y., Zhang, W., & Yin, L. (2024). Contrastive Diffuser: Planning Towards High Return States via Contrastive Learning arXiv:2402.02772.

Tang, C., Abbatematteo, B., Hu, J., Chandra, R., Martín-Martín, R., & Stone, P. (2025). Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes. *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 39, pp. 28694–28698)

Wilcox, A., Balakrishna, A., Dedieu, J., Benslimane, W., Brown, D., & Goldberg, K. (2022). Monte Carlo Augmented Actor-Critic for Sparse Reward Deep Reinforcement Learning from Suboptimal Demonstrations. *Advances in Neural Information Processing Systems, 35*, 2254–2267.

Yang, J., Zhu, B., Chen, J., & Jiang, Y.-G. (2026). Actor-critic for continuous action chunks: A reinforcement learning framework for long-horizon robotic manipulation with sparse reward. *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 40, pp. 18692–18700)

Zheng, C., Eysenbach, B., Walke, H., Yin, P., Fang, K., Salakhutdinov, R., & Levine, S. (2023). Stabilizing Contrastive RL: Techniques for Robotic Goal Reaching from Offline Data arXiv:2306.03346.

Zhu, J., Xia, Y., Wu, L., Deng, J., Zhou, W., Qin, T., Liu, T.-Y., & Li, H. (2022). Masked Contrastive Representation Learning for Reinforcement Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 45*(3), 3421–3433.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Haotian Zhang[1] · Chen Wang[1] · Jun Moon[1]

✉ Jun Moon
junmoon@hanyang.ac.kr

Haotian Zhang
zht771360327@hanyang.ac.kr

Chen Wang
wcgyy@hanyang.ac.kr

[1] Department of Electrical Engineering, Hanyang University, Seoul 04763, South Korea